

AI Tools for Economics Research

Session 4 — Advanced use

Tuesday 23 June 2026

10:30–12:00 · 14:30–16:00

Banco de Portugal

João B. Sousa · Nova SBE

TODAY

Customising the agent for recurring research workflows.

So far we've used the agent off-the-shelf. Today we configure it — for one user, then for a team.

01

Skills.

THE PROBLEM

You keep typing the same instructions.

Every cleaning job: the same format rules, the same naming, the same stop-and-ask. Re-typing them each session is error-prone and easy to forget under time pressure.

`CLAUDE.md` holds standing conventions. A skill packages a whole procedure.

SKILLS

A named, reusable procedure.

A folder with a markdown file: a description of when to use it, and the steps to follow. The agent loads it when the task matches, or you invoke it by name with `/name`.

Lives in the repo. Shared via git, like `CLAUDE.md`.

```
.claude/skills/  
└─ clean-series/  
   └─ SKILL.md
```

```
---
```

```
name: clean-series  
description: Align a raw  
             series to monthly YoY,  
             long format, with checks.
```

```
---
```

1. Read the raw CSV.
2. Confirm the column.
3. Compute YoY.
4. Write to processed/.

EXAMPLE 1

A dataset-cleaning skill.

Encode Friday's data-cleaning procedure once: confirm the column, align frequency, compute YoY, write long-format to `processed/`, report missing-value counts. Next series, you type `/clean-series` instead of the full prompt.

The verification steps travel with the skill — applied consistently every time.

EXAMPLE 2

A memo-drafting skill.

Default format for a results memo: section order, table format, the rule that every number cites its source file and line. Invoke it on a folder of outputs; get a draft in the same shape every time.

Consistency across reports,
without restating the style guide
in every prompt.

WHICH TOOL

Prompt, CLAUDE.md, or skill?

Prompt one-off, this turn only.

CLAUDE.md a standing convention that holds every turn ("data/raw is read-only").

Skill a multi-step procedure you repeat across sessions ("clean a series", "draft a memo").

Rule of thumb: if you've typed the same steps twice, write a skill.

02

Hooks.

HOOKS

Scripts the harness runs, not the model.

A skill is instructions the model *may* follow. A hook is a command the harness — the program running the agent — *always* runs, on a set event, with no model involvement.

Configured in `settings.json`.

The action is any command you can run in your shell.

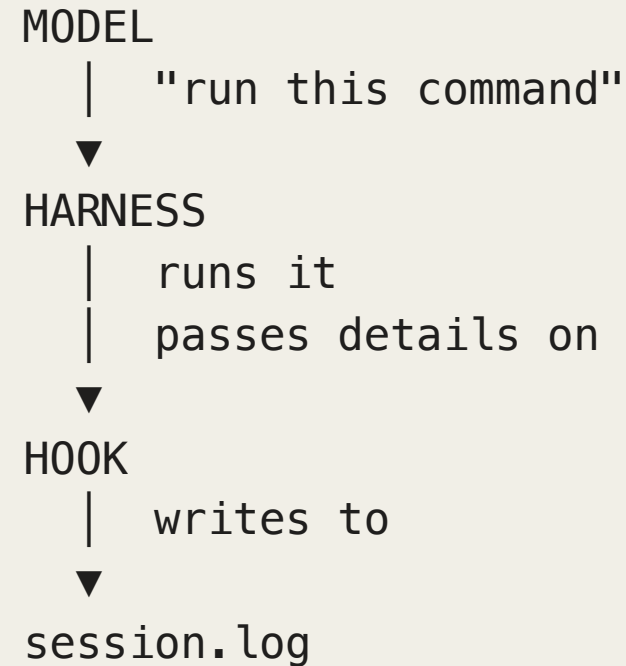
events

PreToolUse	before a tool runs (can block it)
PostToolUse	after a tool runs
Stop	when the agent finishes a turn

HOW A HOOK KNOWS WHAT HAPPENED

The harness hands the work to the hook.

1. The model asks to run a command.
2. The harness runs it.
3. The harness passes the details on to your hook.
4. The hook does its job — here, writes the command to a log.



Logging, for reproducibility.

Audit trail append every bash command to `session.log`.

Cost tracking log token counts per session to a CSV.

Edit diary save a record of what changed after every file edit.

Data-access log record every read of `data/raw/` with a timestamp.

AN AUDIT-TRAIL HOOK

`.claude/settings.json`

```
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Bash",
      "hooks": [{
        "type": "command",
        "command": "jq -r '.tool_input.command' >> session.log"
      }]
    }]
  }
}
```

Every bash command the agent runs lands in `session.log`.

SAME MECHANISM, OTHER TRIGGERS

Checks and formatting are hooks too.

PreToolUse before a write, run the formatter
— or block an edit to a protected path.

Pre-commit checks run the test suite before a
commit is allowed.

Guardrails refuse any command that edits
`data/raw/` in place.

A hook can block the action, not
just observe it. This is how you
enforce `CLAUDE.md` rather than
only request it.

Where we are.

1. **Skills package a procedure** — a named, multi-step workflow the agent runs on demand, shared via git.
2. **Hooks run deterministically on events** — no model involvement; they observe or block.
3. **Together they move conventions from "asked" to "enforced"** — the difference between `CLAUDE.md` and a hook.

Next: subagents, MCP, and a shared team setup.

BREAK

14:30

Back after lunch.

03 · AFTERNOON

Subagents.

SUBAGENTS

A delegated task with its own context.

The main agent spawns a subagent, hands it one task, and gets back only the result. The subagent's context — the files it read, the dead ends — never enters the main session.

Protects the main context from the noise of a search.

```
main session
├── subagent: search
│   ├── reads 40 files
│   └── returns 1 answer
└── context stays clean
```

WHY ITS OWN CONTEXT MATTERS

The reading happens elsewhere.

Each subagent has a separate context window. It reads the source there — and only its short answer crosses back. The main agent's context fills with results, not raw material.

stays in the subagent
the full abstracts
the files it opened
the dead ends

crosses back
one table

WHY DELEGATE

Three reasons.

Protect context a wide search reads many files;
only the conclusion comes back.

Parallelism independent subtasks run at once
— search three folders simultaneously.

Focus a subagent with one job and a narrow
tool set is harder to derail.

And when it doesn't.

Helps broad read-only search, parallel tasks, anything that floods the main context.

Doesn't a small task — the overhead exceeds the work.

Doesn't sequential dependence — if step 2 needs step 1's reasoning, keep them together.

LIVE DEMO

Literature triage, parallelised.

Nine abstracts in three topic folders.
One subagent per folder, each
returning a *question · method · data ·
finding* table. The main session
merges three tables — the nine
abstracts never enter it.

Each subagent's misreads stay
local.

```
papers/litreview/  
├─ monetary/  }  
├─ fiscal/    } one subagent  
└─ labour/    } each → a table
```

```
main session  
  merges 3 tables,  
  not 9 abstracts
```

LIVE DEMO

1. Single agent, one context:

```
Read every abstract under papers/litreview/  
and build one comparison table.
```

2. One subagent per folder:

```
For each folder in papers/litreview/, spawn a  
subagent that reads the abstracts and returns a  
table – columns: question · method · data · finding.  
Then merge the three tables into one.
```

Same corpus, both prompts.
Compare the main session's
context after each.

04

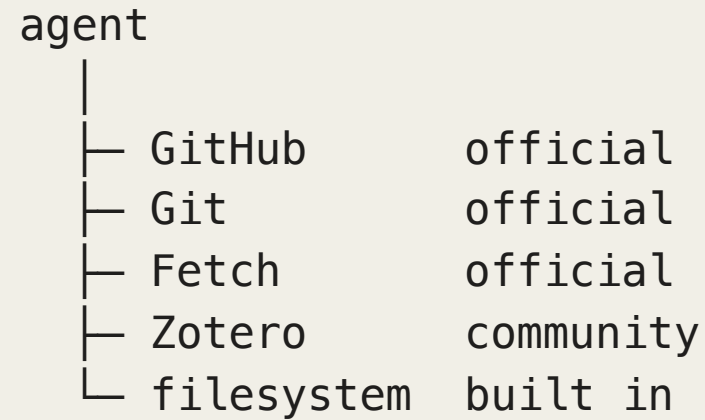
MCP and external tools.

MCP

A standard plug for external tools.

Model Context Protocol: a common interface between the agent and an outside service. Install a server once; its tools become available to the agent, like the built-in file and shell tools.

One protocol, many servers —
the agent needs no bespoke
code per service.



USEFUL SERVERS

For a research workflow — all community-built.

Reference manager Zotero — search your library, pull citation keys, stage new references.

Data sources pull a FRED or Eurostat series by code, without leaving the session.

Paper search query Google Scholar or Semantic Scholar for the literature pass.

None of these is an official vendor server — they are third-party projects. See the next slide before connecting one.

Two things to weigh.

Trust a server runs code and can send your data to a third party; one that fetches outside content can carry hidden instructions that try to hijack the agent. Read what it does before connecting.

Clutter connect the servers you use, not every one available — each adds tools the agent has to weigh.

05

A small team
workflow.

The configuration is just files in the repo.

CLAUDE.md shared conventions — paths, tools, house style.

Skills the team's standard procedures — clean a series, draft a memo.

Hooks the team's guardrails — audit log, protected data, pre-commit tests.

THE PATH

Start with one file. Grow from use.

Begin with a `CLAUDE.md`. When you've typed the same steps twice, write a skill. When a convention must hold without exception, write a hook. Commit each. The shared config is the common workflow of the team, in version control.

Three ideas to conclude.

1. **Skills and hooks turn one-off prompts into shared infrastructure** — procedures you invoke, guardrails that always run.
2. **Subagents handle width** — broad search and parallel work, with the main context kept clean.
3. **The whole setup is files in git** — `CLAUDE.md`, skills, hooks.
Clone the repo, inherit the workflow.

Thank you.

joao.sousa@novasbe.pt